Python-List

Objective

- Concept of List and Basic Operation
  - Create
  - Insert
  - Update
  - Delete
  - Iterate
- Basic List Operator
- Built-in List Functions
- List methods
- Joining Lists and Splitting Strings
- List Comprehension

Concept of List

- The list is a most versatile datatype available in Python which can be written as a list of comma-separated values (items) between **square brackets**.
- Important thing about a list is that items in a list **need not be of the same type**.

Characteristics of Lists

1. The lists are ordered.
2. The element of the list can access by index.
3. The lists are the mutable type.
4. A list can store the number of various elements.

Creating a list

- Creating a list is as simple as putting different comma-separated values between square brackets.
- It can have any number of items and they may be of different types (integer, float, string etc.).
- Similar to string indices, list indices start at 0, and lists can be sliced, concatenated and so on.

```
# list with mixed data types
list1 = ['physics', 'chemistry', 1997, 2000];

list2 = [1, 2, 3, 4, 5 ]; # list of integers
list3 = ["a", "b", "c", "d"]

my_list = [] # empty list

# nested list
my_list = ["mouse", [8, 4, 6], ['a']]
```


www.object.co.in

Accessing Values in Lists

OBJECT™ TECHNOLOGIES

- We can use the index operator [] to access an item in a list. In Python, indices start at 0. So, a list having 5 elements will have an index from 0 to 4.
- Trying to access indexes other than these will raise an IndexError.
- The index must be an integer. We can't use float or other types, this will result in TypeError.
- Nested lists are accessed using nested indexing.

Example

```
.# List indexing
my_list = ['p', 'r', 'o', 'b', 'e']
print(my_list[0]) # Output: p
print(my_list[2]) # Output: o
print(my_list[4]) # Output: e

# Nested List
n_list = ["Happy", [2, 0, 1, 5]]

# Nested indexing
print(n_list[0][1]) # Output: a
print(n_list[1][3]) # Output: 5

# Error! Only integer can be used for
indexing
print(my_list[4.0])
```

Output:

```
p
o
e
a
5
Traceback (most recent call last):
  File "<string>", line 21, in
<module>
TypeError: list indices must be
integers or slices, not float
```

Updating List values

- Lists values can be updated by using the **slice and assignment operator.**
- Python also provides append() and insert() methods, which can be used to add values to the list.

```
list = [1, 2, 3, 4, 5, 6]
print(list)

list[2] = 10   # It will assign value to the value to the second index
print(list)

list[1:3] = [89, 78]     # Adding multiple-element
print(list)

list[-1] = 25  # It will add value at the end of the list
print(list)
```

Output:

```
[1, 2, 3, 4, 5, 6]
[1, 2, 10, 4, 5, 6]
[1, 89, 78, 4, 5, 6]
[1, 89, 78, 4, 5, 25]
```

www.object.co.in

Accessing Value by list slice

```
mylist= [1, 89, 78, 4, 5, 25]
list2= mylist[:]
print(list2[-1])
print(list2[::-1])
print(list2[-3:-1])
print(list2[1:3])
print(list2)
```

Output:

```
25
[25, 5, 4, 78, 89, 1]
[4, 5]
[89, 78]
[1, 89, 78, 4, 5, 25]
```


www.object.co.in

Delete List Elements

The del keyword removes the specified index:

```
thislist = ["apple", "banana", "cherry"]
del thislist[0]
print(thislist)
```

output:

```
['banana', 'cherry']
```

The del keyword can also delete the list completely:

```
thislist = ["apple", "banana", "cherry"]
del thislist
print(thislist)
```

output:

```
Traceback (most recent call last):
  File "C:UsersArnavDesktopPythonsecond.py", line 3, in <module>
    print(thislist)
NameError: name 'thislist' is not defined
```

Iteration

The for loop is used to iterate over the list elements.

```
L1=[ 1,2,3,4]
for i in L1:
    print(i)
```

Output

```
1
2
3
4
```

Basic List Operator

L1=[1,2,3,4]
L2=[5,6,7,8]

Repetition

The repetition operator enables the list elements to be repeated multiple times.

L1*2 => [1, 2, 3, 4, 1, 2, 3, 4]

Concatenation

It concatenates the list mentioned on either side of the operator.

L1+L2 => [1, 2, 3, 4, 5, 6, 7, 8]

Membership

It returns true if a particular item exists in a particular list otherwise false.

print(2 in L1) #Output will be True.

www.object.co.in

Built-in List Functions

1. **len(list)**
   Gives the total length of the list.

2. **max(list)**
   Returns item from the list with max value.

3. **min(list)**
   Returns item from the list with min value.

4. **list(seq)**
   Converts a any sequence (tuple,string,...) into list.

List methods

- **append**

  Add a value to the end of a list.

```
x = [1, 2, 3]
x.append(4) # x is now [1, 2, 3, 4]
```

- **index**

  Return the index of an item in a list.

```
x = [1, 2, 3, 5, 2, 4]
y = x.index(3) # y is now 2
```

- **pop**

  Remove an item from a list and return it.

```
x = [1, 2, 3, 4]
y = x.pop(1) # x is now [1, 3, 4]; y is now 2
z = x.pop() # x is now [1, 3]; z is now 4
```


www.object.co.in

- **remove**

  Remove a specified element from a list.

```
x = [1, 2, 3, 4]
x.remove(2) # x is now [1, 3, 4]
```

- **Reverse**

  Reverse the order of a list

```
x = [1, 2, 3, 4]
x.reverse() # x is now [4, 3, 2, 1]
```

- **sort**

  Sort a list in numeric or alphabetic order

```
x = [3, 1, 4, 2]
x.sort() # x is now [1, 2, 3, 4]
y = ['c', 'd', 'b', 'a']
y.sort() # y is now ['a', 'b', 'c', 'd']
```

count

Returns the number of elements with the specified value i.e it will Returns count of how many times obj occurs in list

```
aList = [123, 'xyz', 'zara', 'abc', 123];
print ("Count for 123 : ", aList.count(123))
print ("Count for zara : ", aList.count('zara’))
```

Output:

Count for 123 : 2

Count for zara : 1

insert

This method does not return any value but it inserts the given element at the given index.

list.insert(index, obj)

index - This is the Index where the object obj need to be inserted.

obj - This is the Object to be inserted into the given list.

```
list1 = ["apple", "banana", "cherry"]
list1.insert(2,"Mango")
print(list1)
```

Output:

['apple', 'banana', 'Mango', 'cherry']

Joining Lists and Splitting Strings

OBJECT TECHNOLOGIES™

Python String split() Method

string.split(separator, maxsplit)

separator :- Optional. Specifies the separator to use when splitting the string. By default any whitespace is a separator
maxsplit :- Optional. Specifies how many splits to do. Default value is -1, which is "all occurrences"

```
txt = "welcome to the jungle"
x = txt.split()
print(x)
```

Output:
['welcome', 'to', 'the', 'jungle']

```
txt = "apple#banana#cherry#orange"
x = txt.split("#")
print(x)
```

Output:
['apple', 'banana', 'cherry', 'orange']

www.object.co.in

Python String join() Method

- The join() method takes all items in an iterable and joins them into one string.
- A string must be specified as the separator.

Syntax

string.join(iterable)

iterable :- Required. Any iterable object where all the returned values are strings

```
myTuple = ("John", "Peter", "Vicky")
x = "#".join(myTuple)
print(x)
```

Output:

John#Peter#Vicky

www.object.co.in

List Comprehension

Rather than creating an empty list and adding each element to the end, you simply define the list and its contents at the same time by using comprehension.

Syntax of List Comprehension

[expression for item in list]

```
x = [x*2 for x in range(11)]
print(x)
```

Output:

[0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20]

Conditionals in List Comprehension

OBJECT TECHNOLOGIES™

List comprehensions can utilize conditional statement to modify existing list (or other tuples).

Example : Using if with List Comprehension

```
number_list = [ x for x in range(20) if x % 2 == 0]
print(number_list)
```

Output:
[0, 2, 4, 6, 8, 10, 12, 14, 16, 18]

```
colours = [ "red", "green", "yellow "]
things = [ "house", "car", "tree" ]
coloured_things = [ (x,y) for x in colours for y in things ]
print (coloured_things)
```

Output:
[('red', 'house'), ('red', 'car'), ('red', 'tree'), ('green', 'house'), ('green', 'car'), ('green', 'tree'), ('yellow', 'house'), ('yellow', 'car'), ('yellow', 'tree')]

www.object.co.i

Summary

- Concept of List and Basic Operation
  - Create
  - Insert
  - Update
  - Delete
  - Iterate
- Basic List Operator
- Built-in List Functions
- List methods
- Joining Lists and Splitting Strings
- List Comprehension

Thank You